Reviewing AI-Generated Code: A Checklist

Step 4 of the review workflow from the Real Python tutorial How to Review AI-Generated Python Code Efficiently. Visit realpython.com to learn more.

How to use: Work through each section for the change under review. For every item, ask the question and scan for the red flag (▸). Tick the box once you've confirmed it's clear.

1 Architecture, Modularization & Fit

- [ ] **Layering** — Is each code piece in the right layer?
  - ▸Logic in the wrong layer, a grab-bag `utils` dumping ground, or circular imports
- [ ] **Modularization** — Is the code split into the right modules and packages?
  - ▸One module doing the work of several, or related code scattered across many files
- [ ] **Cohesion** — Does each module or class do just one thing?
  - ▸A class or module mixing unrelated responsibilities
- [ ] **Fit** — Does it match the project's conventions and patterns?
  - ▸Code that ignores project conventions or differs from the rest of the codebase

2 Logic Bugs

- [ ] **Silently wrong result** — Does the output match intent?
  - ▸Plausible output that's incorrect
- [ ] **Wrong or flipped condition** — Do the branches match the intended logic?
  - ▸Inverted booleans, `and` / `or` mix-ups, `<` vs `<=`, or a missing `else` branch
- [ ] **Wrong number or date math** — Are money and time handled with the right types?
  - ▸Floats used for money, or timezone-naive datetimes
- [ ] **Off-by-one** — Do loop and slice bounds match the intended count?
  - ▸A loop or slice that starts or stops one item off

3 Missing Edge Cases & Swallowed Errors

- [ ] **Missing edge cases** — Does it handle inputs beyond the happy path?
  - ▸No handling for empty, `None`, zero, negative, or oversized inputs
- [ ] **Swallowed errors** — Do failures surface instead of getting silenced?
  - ▸Exceptions caught and ignored, so failures pass in silence

4 Security Weaknesses

- [ ] **Hardcoded secrets** — Are keys and tokens kept out of the source?
  - ▸API keys, passwords, or tokens left in the code
- [ ] **Missing authorization** — Is every sensitive path behind a login and ownership check?
  - ▸A dropped login or ownership check
- [ ] **Unsafe defaults** — Are TLS and hashing set to safe defaults?
  - ▸TLS verification off, or weak hashes like md5 and sha1
- [ ] **Injection** — Is user input kept out of SQL, `eval()`, and `exec()`?
  - ▸SQL built from strings, or `eval()` and `exec()` on user input
- [ ] **Secrets in logs** — Do logs stay free of tokens and payloads?
  - ▸Tokens, passwords, or whole requests written to logs

5 Made-Up APIs & Packages

- [ ] **Nonexistent API** — Does every method and import actually exist?
  - ▸Methods or imports the agent could have invented
- [ ] **Fake packages** — Is every dependency a real, published package?
  - ▸A dependency name the agent hallucinated

6 Performance & Resource Traps

- [ ] **Slow database access** — Do queries stay out of loops and avoid full scans?
  - ▸One query per loop item (N+1), or full-table scans
- [ ] **Slow algorithms** — Do hot paths avoid nested loops over big data?
  - ▸Nested loops over big data, or `O(n²)` on hot paths
- [ ] **Resource leaks** — Is every file or connection closed?
  - ▸Files, sessions, or connections opened but never closed

7 Concurrency & Async Bugs

- [ ] **Race conditions** — Is shared state guarded against concurrent access?
  - ▸Reading then updating shared state without a lock
- [ ] **Unsafe shared state** — Does mutable state cross task or thread boundaries safely?
  - ▸Mutable globals, caches, or instance attributes written from more than one task
- [ ] **Blocking the event loop** — Does async code stay free of blocking calls?
  - ▸Synchronous I/O, `time.sleep()`, or CPU-heavy work inside an `async def` function
- [ ] **Deadlocks** — Are locks always acquired in a consistent order?
  - ▸Nested locks, or a lock held across an `await`

8 Maintainability Problems

- [ ] **Vague names** — Does each name pin down what it holds or does?
  - ▸Generic names like `data`, `result`, or `process()` that don't reveal the concept
- [ ] **Needless abstraction** — Does each layer of indirection earn its keep?
  - ▸Speculative options, wrappers, or generality that nothing uses yet
- [ ] **Semantic duplication** — Is the same logic reimplemented in disguise?
  - ▸The same behavior rewritten differently, so duplicate-code checks miss it
- [ ] **Stale comments** — Do comments and docstrings still match the code?
  - ▸Docstrings or comments describing what the code used to do

Confirm every flag by running the code before you fix it — plausible isn't the same as correct.

Real Python